PennStateSoft Meeting Scheduling System
Design Document
Version 1.0

TEAM MEMBERS

Name	Student ID
Samuel Shumake	sms8868
Victoria Worthington	vkw5048
Lianjie Sun	ljs6051
Tiffany Hart	tph5606

Revision History

Date	Version	Description	Author
<07/26/2026>	1.0	Initial draft of Design Document	Group 1

Table of Contents

Design Document

1. Introduction

This design document will provide all necessary details concerning the design of the PennStateSoft Meeting Scheduling System software. It will include sections devoted to Architecture Design, Software Interface Design, and Internal Module Design. The following sub-sections will cover the purpose, scope, and overview of this document as well as an overview of any definitions and references needed to understand the material.

1.1 Purpose

The purpose of this document is to display the architectural design, provide the class diagrams, and give an outline for the general design of the PennStateSoft Meeting Scheduling System. This design document will serve as a contract for the implementation of the project.

1.2 Scope

This document specifies the Architectural Design (AD), Module Interface Design (MID), and Internal Module Design (IMD) for the PennStateSoft Meeting Scheduling System (PMSS). PMSS is a web-based application designed for PennStateSoft employees to create, manage, and attend meetings, and by administrators to manage rooms, billing, and complaints. This SDD connects to the SRS, translating each functional and nonfunctional requirement into design by defining the software architecture and its rationale, the run-time topology, providing an architectural risk analysis, the interactions between modules, and the classes, attributes, and methods that implement each module. The designs described in this document cover the following capabilities identified in the SRS:

- Clients: account registration, profile viewing and editing, meeting creation and room reservation, paying for special room reservations, adding and removing meeting participants, viewing and editing owned meetings, viewing attended meetings, and filing complaints
- Administrator: add and delete meeting rooms, view and edit client billing information, view and respond to user complaints, and meeting display filters by week, day, room, attendee, and/or timeslot.

Consistent with the SRS, the design is bound by a single-week, business hour (9 AM to 5 PM) based scheduling model supporting one-hour time slots and one room per meeting. Multi-week or recurring planned meetings, designated mobile or desktop applications, and integration with external calendar systems are outside the scope of this design.

1.3 Definitions, Acronyms, and Abbreviations

Acronym/Abbreviation	Definition
AD	Architectural Design
IMD	Internal Module Design
MID	Module Interface Design
MVC	Model-View-Controller
PMSS	PennStateSoft Meeting Scheduling System

1.4 References

This subsection provides a complete list of documents referenced through this SDD.

1. MITRE Corporation. Common Weakness Enumeration (CWE).
   Referenced throughout Section 2.4, Architectural Risk Analysis: CWE-20, 79, 89, 117, 200, 269, 287, 307, 328, 362, 384, 434, 476, 521, 532, 537, 640, 767, 778, 840, 862, 863.
   Available at: https://cwe.mitre.org/data/definitions/ (append the CWE number, for example .../20.html for CWE-20).
2. MITRE Corporation. Common Attack Pattern Enumeration and Classification (CAPEC), CAPEC-7: Blind SQL injection.
   Referenced in Section 2.4.3.4, Threat and Vulernability Mapping.

Available at: https://capec.mitre.org/data/definitions/7.html.
3. CERT. (2007). Common Sense Guide to Prevention and Detection of Insider Threads. Software Engineering Institute, Carnegie Mellon University.
   Referenced in Section 2.4.3.2, Threat Analysis.
4. Group 1, 2026. PennStateSoft Meeting Scheduling System: Software Requirements Specification (SRS). Internal document.
   Referenced throughout this SDD as the source of the functional and nonfunctional requirements translated into design.

1.5 Overview

The SDD is organized into four sections. Section 1 introduces high-level introductions into the PMSS which includes the system's purpose, scope, definitions, and references provided for the document. Section 2 introduces the architectural design of the PMSS where the architectural design is discussed, including the software architecture diagram, topology, and risk analysis are discussed. Section 3 introduces the interface design including the user, software, and hardware interfaces, the module interface diagrams, and the dynamic models of system interfaces. Section 4 provides the detailed internal module design, includes the module class diagrams and their accompanying class charts that describe each class's attribute, methods, visibility, return types, and responsibilities.

2. Architectural Design

2.1 Rationale

PMSS uses a Model-View-Controller (MVC) architectural pattern to separate the PMSS Server into Controller and Model components with a client/administrator View. The MVC model was chosen for its separation of logic, data, and encapsulation, supporting the maintainability, flexibility, and reusability quality requirements of the SRS. It also enforces a strict security boundary: the View interacts with the system only through the Controller through the internet, so all authentication, validation, and role-based access control must happen before a request can reach the Model or Database. This address external threats, such as crackers attacking over the internet, at the controller level, while insider threads with direct LAN or Database access are addressed separately through data-layer controls, such as encryption and least-privilege access.

2.2 Software Architecture Diagram

The following is a high-level architectural diagram of the PennStateSoft Meeting Scheduling System. The software is split into six packages that follow the Model-View-Controller paradigm, each of which has a relationship with the database. Many of the packages mix-and-match their models such as the User class being used in essentially every package.

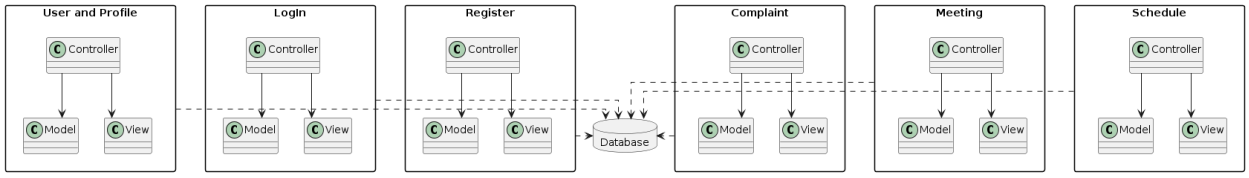

Figure 1: Software Architecture Diagram

2.3 System Topology

The following is a high-level component diagram detailing the topology of the system. It includes both Client and Administrator PCs that connect to the Web Server. The Web Server contains the packages outlined in Section 4, which in turn connect to the database containing all of the stored data for the software.

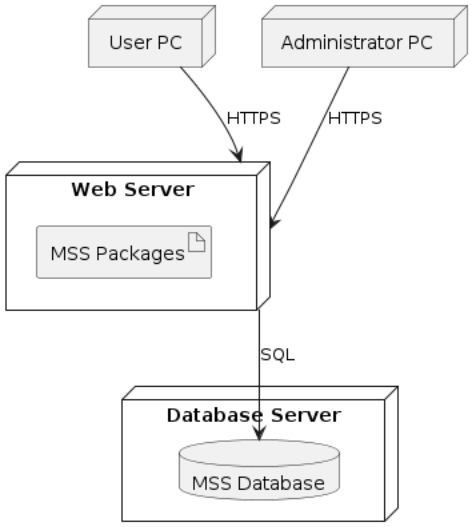

Figure 2: System Topology

2.4 Architectural Risk Analysis

2.4.1 Software characterization

The Software Characterization diagram illustrates the architect of the PennStateSoft Meeting Scheduling System (PMSS) showing the interactions between the users, Clients and Administrators, the presentation layer (MVC), the PMSS server, and the database. It identifies the internet and LAN trust boundaries and highlights the locations where internal and external threats may interact with the system.

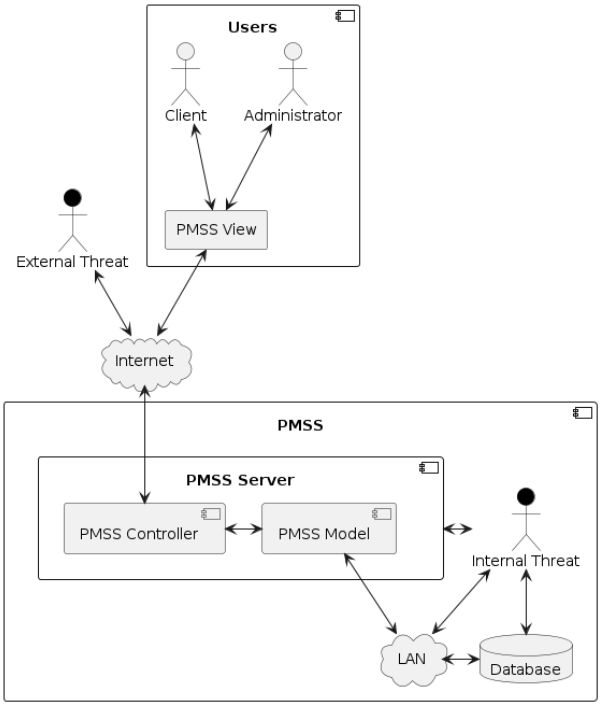

Figure 3: Software Architecture Diagram

2.4.1 Threat Analysis

The Threat Analysis chart identifies the main threat sources to the PMSS, their motivations, and the possible threat actions they may take against the system.

Threat Source	Motivation	Threat Actions
Cracker	1. Challenge 2. Ego 3. Rebellion	• Social egngineering • System Intrusion • System profiling • Unauthorized System Access
Computer Criminal	1. Destruction of Information 2. Information Disclosure 3. Monetary Gain 4. Unauthorized data alteration	• Computer crime • Fraudulent act • Information Bribery • Malware: Trojans, Worms, Virus, spyware • Ransomware • Spam, Phishing, Baiting, Drive-by-Download • Spoofing • System intrusion
Industrial espionage	1. Blackmail 2. Competitive Advantage 3. Economic Espionage	• Economic exploitation • Information theft • Intrusion on personal privacy • Social Engineering • System Penetration • Unauthorized System Access
Insiders (poorly trained, disgruntled, malicious, negligent, dishonest, or terminated employees) [CERT 2007]	1. Curiosity 2. Ego 3. Intelligence 4. Lack of procedures or training 5. Monetary gain 6. Revenge 7. Unintentional errors and omissions 8. Wanting to help the company	• Assault on an employee • Blackmail • Browsing of proprietary information • Computer abuse • Fraud and theft • Information bribery • Input of falsified information • Interception • Malicious code • Sale of personal information • System bugs • System intrusion • System sabotage • Unauthorized access

2.4.2 Architectural Vulnerability Assessment

2.4.2.1 Attack Resistance Analysis

2.4.2.1.1 Input Validation

The system will check all user input before processing. The input fields in the login, registration, profile, and meeting scheduling dashboards and will be checked to ensure correct formatting and does not contain invalid data

Reference: MITRE CWE-20: Improper Input Validation - https://cwe.mitre.org/data/definitions/20.html

2.4.2.1.2 Cross-Site Scripting

The system will help prevent Cross-Site Scripting by validating user input before it is displayed. This will prevent malicious scripts from running in the user's browser.
Reference: MITRE CWE-79: Cross-Site Scripting - https://cwe.mitre.org/data/definitions/79.html

2.4.2.1.3 SQL Injection

The system will use parameterized queries (placeholders) instead of embedded user input when accessing the database. User input will be validated prior to being sent to the database to reduce the risk of SQL injection.
Reference: MITRE CWE-89: SQL Injection - https://cwe.mitre.org/data/definitions/89.html

2.4.2.1.4 Exposure of Sensitive Information

The system will protect sensitive information by storing passwords as hashed values and limiting access to personal data. Secure connections will be used when the data is sent between the client and the server.
Reference: MITRE CWE-200: Exposure of Sensitive Information - https://cwe.mitre.org/data/definitions/200.html

2.4.2.1.5 Improper Privilege Management

The system will use role-based access control, meaning clients and administrators will only be able to access the features that match their assigned roles.
Reference: MITRE CWE-269: Improper Privilege Management - https://cwe.mitre.org/data/definitions/269.html

2.4.2.1.6 Improper Authentication

The system will require users to log in with a valid email and password before accessing the system. Passwords will be verified using their stored password hashes.
Reference: MITRE CWE-287: Improper Authentication - https://cwe.mitre.org/data/definitions/287.html

2.4.2.1.7 Improper Restriction of Excessive Authentication Attempts

The system will limit the number of failed login attempts to 3. If there are too many incorrect password attempts entered the account will be temporarily locked.
Reference: MITRE CWE-307: Improper Restriction of Excessive Authentication Attempts - https://cwe.mitre.org/data/definitions/307.html

2.4.2.1.8 Race Condition

The system will verify that the meeting room is available before confirming a reservation. This will prevent two users from booking the same room at the same time.
Reference: MITRE CWE-362: Race Condition - https://cwe.mitre.org/data/definitions/362.html

2.4.2.1.9 Session Fixation

The system will create a new session ID after a user logs in and remove the session when the user logs out. This will protect the user session from being reused by attackers.
Reference: MITRE CWE-384: Session Fixation - https://cwe.mitre.org/data/definitions/384.html

2.4.2.1.10 Unrestricted Upload of File with Dangerous Type

The system will allow file uploads via profile images, but it will only accept approved file types and reject all others. The uploaded files will also be checked before they are stored.

Reference: MITRE CWE-434: Unrestricted Upload of File with Dangerous Type - https://cwe.mitre.org/data/definitions/434.html

2.4.2.1.11 Weak Password Requirements

The system will require users to create strong passwords that meet a minimum length requirement and complexity requirements to make a password harder to guess.
Reference: MITRE CWE-521: Weak Password Requirements - https://cwe.mitre.org/data/definitions/521.html

2.4.2.1.12 Weak Password Recovery

The system will verify the user's identity before allowing a password reset.
Reference: MITRE CWE-640: Weak Password Recovery - https://cwe.mitre.org/data/definitions/640.html

2.4.2.1.13 Business Logic Errors

The system will check that the users follow the correct steps when creating, editing, or canceling meetings. This is to prevent users from performing actions they should not be allowed to do.
Reference: MITRE CWE-840: Business Logic Errors - https://cwe.mitre.org/data/definitions/840.html

2.4.2.1.14 Missing Authorization

The system will check a user's permissions before allowing access to protected features. The users without permission will be denied access.
Reference: MITRE CWE-862: Missing Authorization - https://cwe.mitre.org/data/definitions/862.html

2.4.2.1.15 Incorrect Authorization

The system will ensure that users can only access their own information and features that match their assigned role. Clients will not have access to administrative functions.
Reference: MITRE CWE-863: Incorrect Authorization - https://cwe.mitre.org/data/definitions/863.html

2.4.2.2 Ambiguity Analysis

2.4.2.2.1 Authorization

The system shall enforce role-based access control. Only administrators shall be permitted to manage user accounts, complaints, billing, and meeting rooms. Clients shall only be permitted to access and modify their own profile information, create and manage their own meeting reservations, and submit complaints.

2.4.2.2.2 Authentication on Login

The system shall require users to authenticate before accessing the PMSS. The design defines that users must provide a valid employee email address and password. Passwords shall be stored as hashed values.

2.4.2.2.3 Error Handling

The system shall display generic error messages during login failures instead of revealing whether the email address or password is incorrect. The system shall display generic error messages when a complaint submission fails. The system shall display generic error messages when meeting creation fails.

2.4.2.2.4 Input Validation

The system shall validate all user input before processing or storing it. Input fields such as registration, login, profile updates, and meeting scheduling define acceptable formats and required fields, reducing uncertainty in how input should be handled. The system shall use parameterized queries (placeholders) for all database operations involving user input to prevent SQL injection attacks.

2.4.2.2.5 Login Failure

The system shall lock the user's account after 3 consecutive failed login attempts within a 15-minute period. The account shall remain locked for 30 minutes or until unlocked by an administrator.

2.4.2.2.6 Meeting Scheduling

The system shall verify that a meeting room is available before confirming a reservation. The system shall reject any reservation request that conflicts with an existing reservation for the same meeting room during the requested date and time.

2.4.2.2.7 Session Management

The system shall generate a new session ID upon successful user authentication. The system shall invalidate the session ID immediately when the user logs out.

2.4.2.3 Dependency Analysis

2.4.2.3.1 Java

CWE-476: NULL Pointer Dereference
A null object reference may cause the application to crash if it is not properly handled.

CWE-537: Information Exposure Through Java Runtime Error Message
Unhandled exceptions may expose implementation details that could assist an attacker.

CWE-767: Access to Critical Private Variable via Public Method
Improper encapsulation may allow critical private variables to be modified through public methods.

2.4.2.3.2 Relational Database Management System

CWE-89: SQL Injection
Improper neutralization of user input may allow attackers to execute unauthorized SQL commands.

CWE-200: Exposure of Sensitive Information
Improper database access controls may expose confidential user information.

CAPEC-7: Blind SQL Injection
Attackers may infer database contents without receiving direct query results.

2.4.2.3.3 Password Hashing Library

CWE-328: Use of a Weak Hash
Using a weak hashing algorithm could allow attackers to recover stored passwords.

2.4.2.3.4 Web Server/Application Server

CWE-384: Session Fixation
Improper session handling could allow an attacker to hijack a user's authenticated session.

2.4.2.3.5 Application Logging

CWE-117: Improper Output Neutralization for Logs
Attackers may inject malicious characters into log data causing log entries to be modified, misinterpreted, or falsified during analysis

CWE- 532: Insertion of Sensitive Information into Log File
Sensitive information written to log files may be exposed if unauthorized users gain access to the logs.

CWE-778: Insufficient Logging
Failure to record important security events or omitting important details about the event when logging it.

2.4.2.4 Threat and Vulnerability Mapping

The Threat and Vulnerability Mapping chart maps the identified threats and references their related software vulnerabilities as discussed in section 2.4.2.

Threat	Vulnerability
Admin Account Takeover	CWE-287: Improper Authentication CWE-307: Improper Restriction of Excessive Authentication Attempts CWE-328: Use of a Weak Hash CWE-521: Weak Password Requirements CWE-640: Weak Password Recovery
Audit Failure	CWE-778: Insufficient Logging CWE-117: Improper Output Neutralization for Logs CWE- 532: Insertion of Sensitive Information into Log File
Complaint/Meeting Stored XSS	CWE-79: Cross-Site Scripting CWE-537: Information Exposure Through Java Runtime Error Message
Double Booking (Race Condition)	CWE-362: Race Condition CWE-476: NULL Pointer Dereference
Malicious File Upload	CWE-434: Unrestricted Upload of File with Dangerous Type
Malformed Input Exploitation	CWE-20: Improper Input Validation
Privilege Escalation	CWE-269: Improper Privilege Management
Special Room Payment Bypass	CWE-840: Business Logic Errors CWE-200: Exposure of Sensitive Information
SQL Injection	CWE-89: SQL Injection

	CWE-767: Access to Critical Private Variable via Public Method CAPEC-7: Blind SQL Injection
Unauthorized Schedule Access	CWE-767: Access to Critical Private Variable via Public Method CWE-384: Session Fixation CWE-862: Missing Authorization CWE-863: Incorrect Authorization

2.4.2.5 Risk Likelihood Determination

2.4.2.5.1 Admin Account Takeover

Motivation and Capacity: Attackers may have medium motivation because administrators accounts provide access to sensitive system functions.
Vulnerability's Impact: High impact because attackers could modify users, rooms, billing information, and complaints.
Effectiveness of Current Controls: Strong passwords, password hashing, and account lockouts reduce risks.
Likelihood: Medium

2.4.2.5.2 Audit Failure

Motivation and Capacity: Attackers may have high motivation because they may attempt to avoid detection by hiding malicious activity.
Vulnerability's Impact: Medium impact because missing logs make attacks harder to identify.
Effectiveness of Current Controls: Logging requirements reduces the risk but depends on proper implementation.
Likelihood: Medium

2.4.2.5.3 Complaint/Meeting Stored XSS

Motivation and Capacity: Attackers may have medium motivation and inject malicious scripts into stored user content.
Vulnerability's Impact: Medium impact because attackers could steal sessions or modify display information.
Effectiveness of Current Controls: Input validation and data cleansing reduces the risk.
Likelihood: Medium

2.4.2.5.4 Double Booking (Race Condition)

Motivation and Capacity: Attackers may have medium motivation and attempt to create conflicting reservations, intentionally or unintentionally.
Vulnerability's Impact: Low impact because it affects scheduling accuracy.
Effectiveness of Current Controls: Availability checks reduce the possibility of conflicting reservations.
Likelihood: Low

2.4.2.5.5 Malicious File Upload

Motivation and Capacity: Attackers may have medium motivation and upload malicious files to compromise the system.
Vulnerability's Impact: High impact because malicious files could affect system security.
Effectiveness of Current Controls: File type validation and upload restrictions reduce risks.
Likelihood: Medium

2.4.2.5.6 Malformed Input Exploitation

Motivation and Capacity: Attackers may have medium motivation and commonly test systems using unexpected input values.
Vulnerability's Impact: Medium impact because it may cause errors or unauthorized behavior.
Effectiveness of Current Controls: Input validation reduces the risks.
Likelihood: Medium

2.4.2.5.7 Privilege Escalation

Motivation and Capacity: Attackers may have high motivation to gain additional permissions.
Vulnerability's Impact: High impact because attackers could access restricted features.
Effectiveness of Current Controls: Role-based access control reduces the risks.
Likelihood: High

2.4.2.5.8 Special Payment Bypass

Motivation and Capacity: Attackers may have medium motivation and may attempt to bypass payment requirements for financial gain.
Vulnerability's Impact: High impact because it could cause financial losses.
Effectiveness of Current Controls: Payment validation reduces the risks.
Likelihood: Medium

2.4.2.5.9 SQL Injection

Motivation and Capacity: Attackers may have high motivation and frequently target SQL injections because automated tools can identify vulnerable systems.
Vulnerability's Impact: High impact because database information could be accessed or modified.
Effectiveness of Current Controls: Parameterized queries (placeholders) and input validation significantly reduce the risk.
Likelihood: Medium

2.4.2.5.10 Unauthorized Schedule Access

Motivation and Capacity: Attackers may have low motivation and attempt to view or modify restricted meeting information.
Vulnerability's Impact: Medium impact because private scheduling information could be exposed.
Effectiveness of Current Controls: Authorization checks and role-based access control reduces the risks.
Likelihood: Low

2.4.2.6 Risk impact determination

2.4.2.6.1 Admin Account Takeover

Threatened Assets: Administrator accounts, billing, room management, complaint management data

Business Impact: Attackers may change important system settings or account sensitive information
Impact Locality: This would most likely affect the administrative privileges of the system.
Impact: High

2.4.2.6.2 Audit Failure

Threatened Assets: Session logs
Business Impact: If logs are incorrect or missing it will be difficult to detect problems or trace attacks.
Impact Locality: This would most likely affect the whole system because all actions should be recorded accurately.
Impact: High

2.4.2.6.3 Complaint/Meeting Stored XSS

Threatened Assets: User accounts, meeting data, complaint forms, session information
Business Impact: Malicious scripts could run in a user's browser(s) and steal data or damage trust in the system
Impact Locality: This would most likely affect users who view the infected content.
Impact: High

2.4.2.6.4 Double Booking (Race Condition)

Threatened Assets: Meeting schedules, room availability, reservation records.
Business Impact: Multiple users could reserve the same room at the same time causing scheduling conflicts.
Impact Locality: This would most likely affect the meeting scheduler class.
Impact: High

2.4.2.6.5 Malicious File Upload

Threatened Assets: User accounts, server files, system integrity.
Business Impact: An attacker could upload malicious files that damage the system or expose data.
Impact Locality: This would most likely affect the file upload feature and possibly the server.
Impact: High

2.4.2.6.6 Malformed Input Exploitation

Threatened Assets: Input fields, database records
Business Impact: Invalid input could cause errors, crashes, or allow attacks on the system.
Impact Locality: This would most likely affect many parts of system, such as the forms and database interactions.
Impact: Medium

2.4.2.6.7 Privilege Escalation

Threatened Assets: Administrative functions, user accounts, protected data.
Business Impact: A non-administrator user could gain features they should not have access to.
Impact Locality: This would most likely affect access control throughout the system.
Impact: High

2.4.2.6.8 Special Payment Bypass

Threatened Assets: Billing records, payment data, account status.
Business Impact: An attacker could avoid paying for a room or change payment related information.
Impact Locality: This would most likely affect the billing and/or payment classes of the system.
Impact: High

2.4.2.6.9 SQL Injection

Threatened Assets: Database records, user accounts, meeting data, complaint data.
Business Impact: An attacker could view, delete, or change data in the database.
Impact Locality: This would most likely affect the database and any functions that interact with it.
Impact: High

2.4.2.6.10 Unauthorized Schedule Access

Threatened Assets: Meeting schedules, user attendee list, user information, reservation details.
Business Impact: A user could view or change meetings they do not have permission to access.
Impact Locality: This would most likely affect the scheduling and profile management system
Impact: High

2.4.2.7 Risk Exposure Statement

The Risk Exposure Statement chart summarizes the identified threats by comparing their Risk Likelihood Determination and Risk Impact Determination to determine their overall Risk Exposure for PMSS.

Threat	Risk Likelihood Determination	Risk Impact Determination	Risk Exposure
Admin Account Takeover	Medium	High	High
Audit Failure	Medium	High	High
Complaint/Meeting Stored XSS	Medium	High	High
Double Booking (Race Condition)	Low	High	Medium
Malicious File Upload	Medium	High	High
Malformed Input Exploitation	Medium	Medium	Medium
Privilege Escalation	High	High	High
Special Room Payment Bypass	Medium	High	High
SQL Injection	Medium	High	High
Unauthorized Schedule Access	Low	High	Medium

2.4.2.8 Risk mitigation planning

2.4.2.8.1 Admin Account Takeover

The system will protect administrator accounts with strong passwords, role-based access control, and account lockouts after too many failed login attempts.

2.4.2.8.2 Audit Failure

The system will keep detailed audit logs so user actions and security events can be tracked and viewed later.

2.4.2.8.3 Complaint/Meeting Stored XSS

The System will validate and escape user input before showing it on the screen to prevent malicious scripts from running.

2.4.2.8.4 Double Booking (Race Condition)

The system will check room availability before saving a reservation to prevent a room from being double booked.

2.4.2.8.5 Malicious File Upload

The system will only allow approved file types and scan uploaded files before storing them.

2.4.2.8.6 Malformed Input Exploitation

The system will check all input for correct format and reject anything invalid before processing it.

2.4.2.8.7 Privilege Escalation

The system will use role-based access control so users can only access the features they’re supposed to use.

2.4.2.8.8 Special Payment Bypass

The system will verify payments before allowing access to special room features or confirming the reservation.

2.4.2.8.9 SQL Injection

The system will use parameterized queries (placeholders) instead of putting user input directly into SQL statements.

2.4.2.8.10 Unauthorized Schedule Access

The system will check permissions before showing or changing schedule information so users can only access their own meetings or allowed data.

3. Software Interface Design

3.1 System Interface Diagrams

3.1.1 User Interface

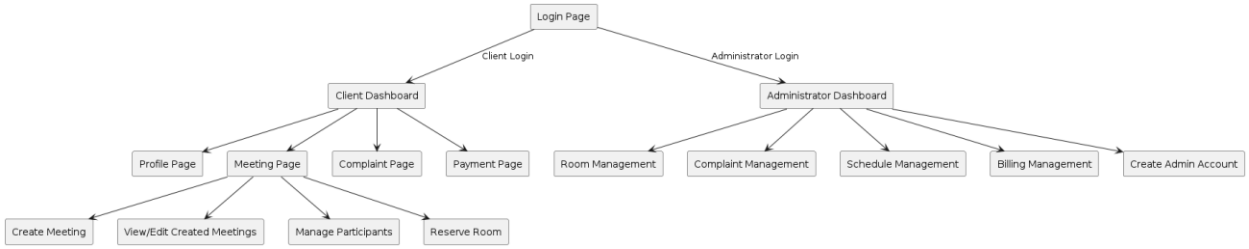

Figure 4: Overall User Interface Diagram

All users begin interaction with the Meeting Scheduling System from the Login page. After successfully authentication, users are redirected to different dashboards according to their user roles.

Clients are redirected to the Client Dashboard, where they can manage their profile information, create and manage meetings, reserve meeting rooms, make payments for special rooms, manage meeting participants, and submit complaints.

Administrators are redirected to the Administrator Dashboard, where they can manage available rooms, update client information, handle complaints, view meeting schedules, and create additional administrator accounts.

The user interface provides a web-based and user-friendly navigation structure that allows users to access required functions without additional training.

3.1.2 Software Interface

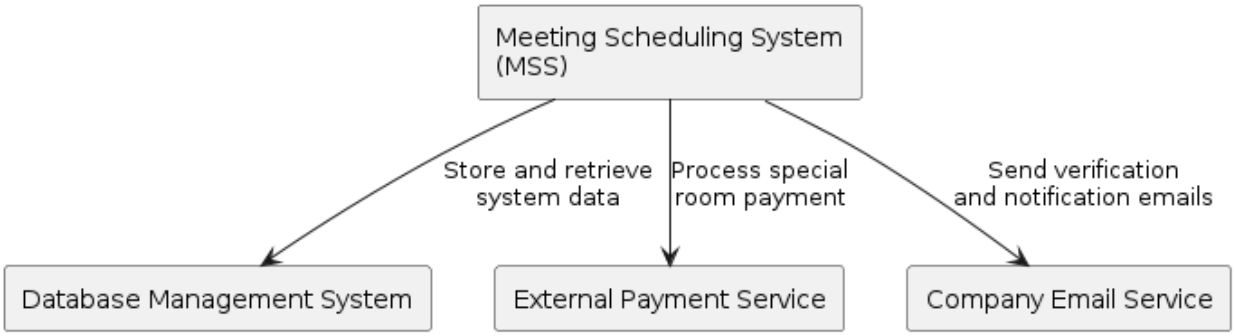

Figure 5: System Software Interface Diagram

The Meeting Scheduling System interacts with several external software systems to provide complete functionality. The system communicates with a database managements system to store and retrieve application data, including user information, meeting schedules, room information, complaints, and payment records.

The system also communicates with an external payment service to process payments required for reserving special rooms. Payment information is transferred securely between the system and the payment service.

In addition, the system interacts with a company email service to support email verification and send system notifications to clients and administrators.

Security Considerations:

Database: Authentication, Access Control, Encrypted Storage

Payment Service: Secure Communication, Payment Verification

Email Service: User Verification, Prevent Unauthorized Messages

3.1.3 Hardware Interface

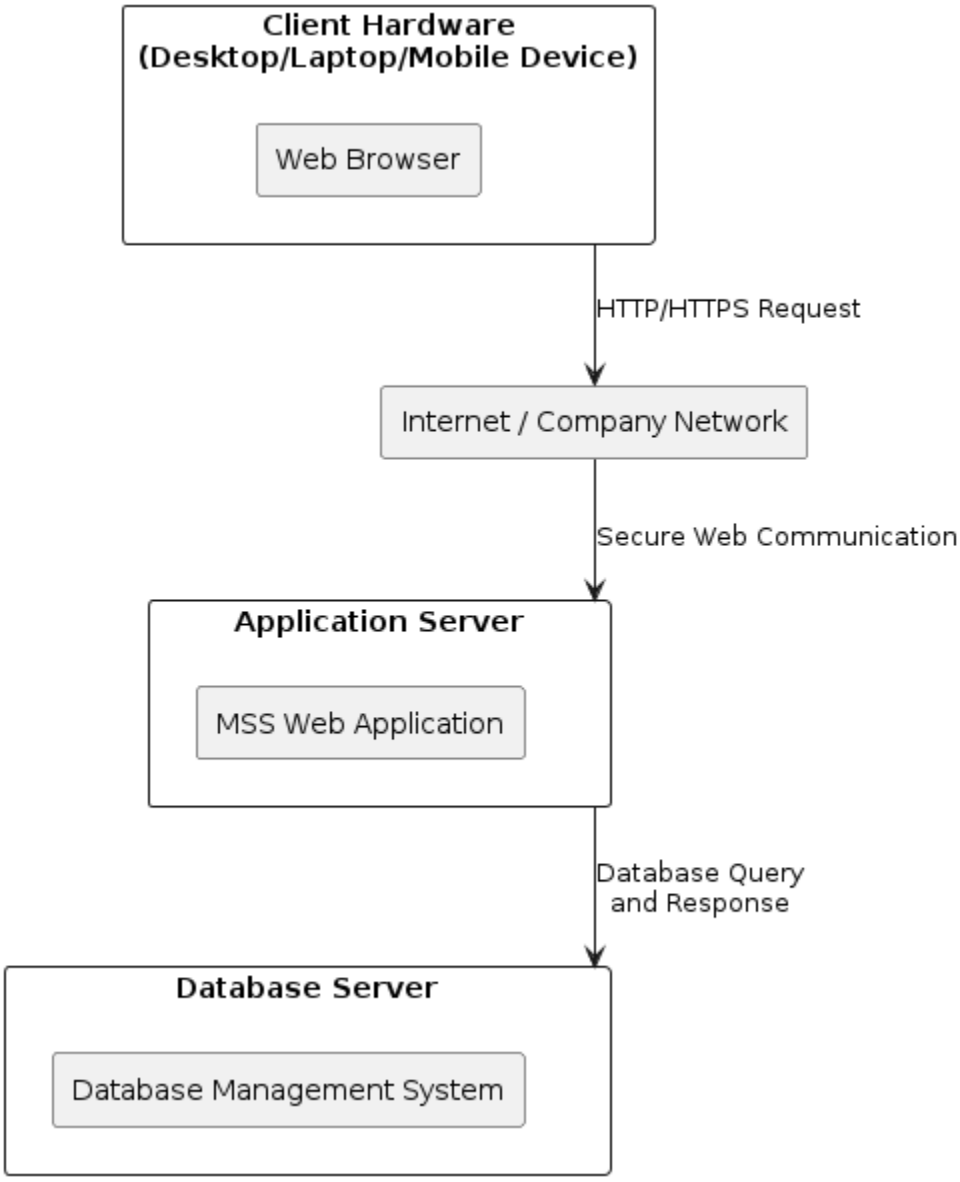

Figure 6: System Hardware Interface Diagram

The Meeting Scheduling System is designed as a web-based application. Users interact with the system through client devices, such as desktop computers, laptops, or mobile devices, using a standard web browser.

Client devices communicate with the MSS application server through a network connection using secure HTTP/HTTPS communication. The application server processes user requests and executes the business logic of the system.

The application server communicates with the database server to store and retrieve system information, including user profiles, meeting schedules, room information, complaints, and payment records.

This hardware architecture provides centralized management, scalability, and secure access for both clients and administrators.

Security Consideration:

Client Device: User Authentication and Session Management

Network Connection: HTTPS Encryption to Protect Transmitted Data

Application Server: Access Control and Server Hardening

Database Server: Authentication, Authorization and Secure Data Storage

3.2 Module Interface Diagrams

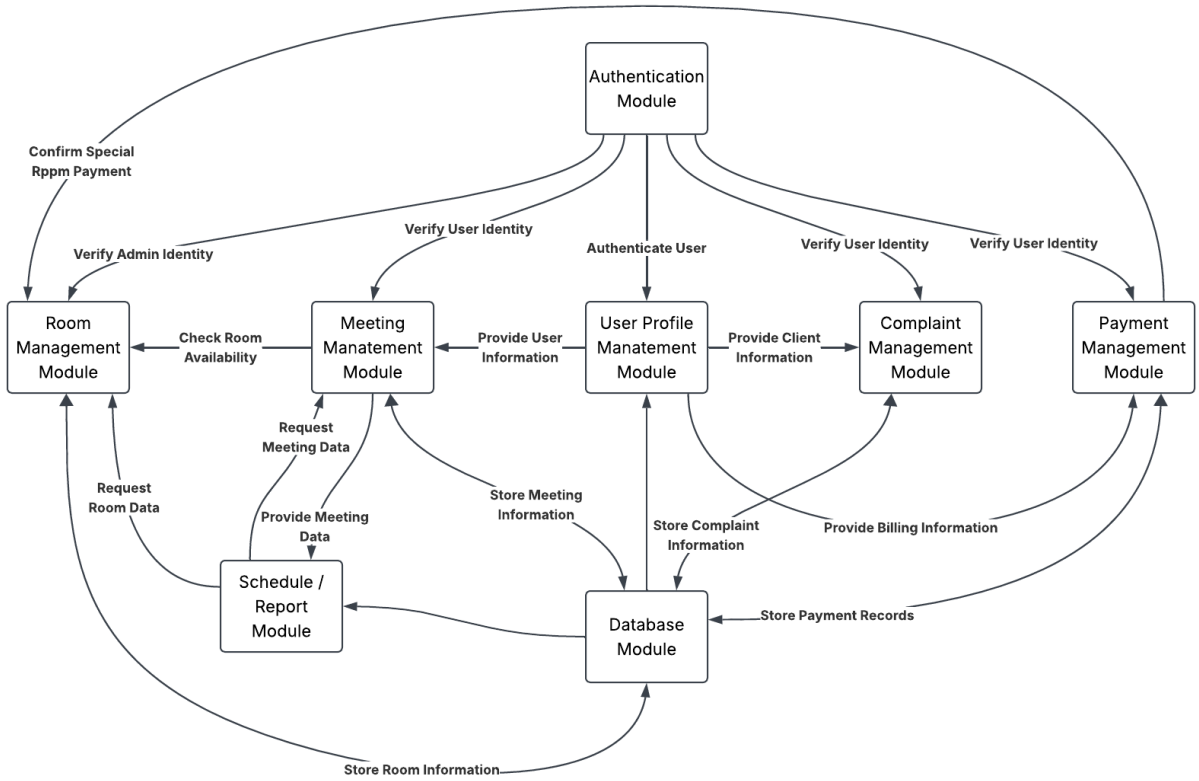

Figure 7: System Module Interface Diagram

The Meeting Scheduling System is divided into several functional modules to improve maintainability, security, and scalability. Each module provides specific services and communicates with other modules through defined interfaces.

The Authentication Module provides user authentication and authorization services for all other modules. The User Profile Management Module maintains user information and provides client data required by meeting, payment, and complaint operations.

The Meeting Management Module is the central module of the system. It communicates with the Room Management Module to verify room availability and interacts with the Schedule/Report Module to provide meeting information. The Payment Management Module communicates with the Room Management Module to verify payments required for special room reservations.

All modules communicate with the Database Module to store and retrieve persistent system information, including user data, meeting schedules, room information, payment records, and complaint records.

Security Consideration:

Authentication Module: Role-Based Access Control and User Authentication

Meeting Module: Authorization Verification Before Modifying Meetings

Payment Module: Secure Payment Processing and Data Encryption

Database Module: Access Control and Secure Data Storage

Complaint Module: User Privacy Protection

3.3 Dynamic Models of System Interface

3.3.1 User Login

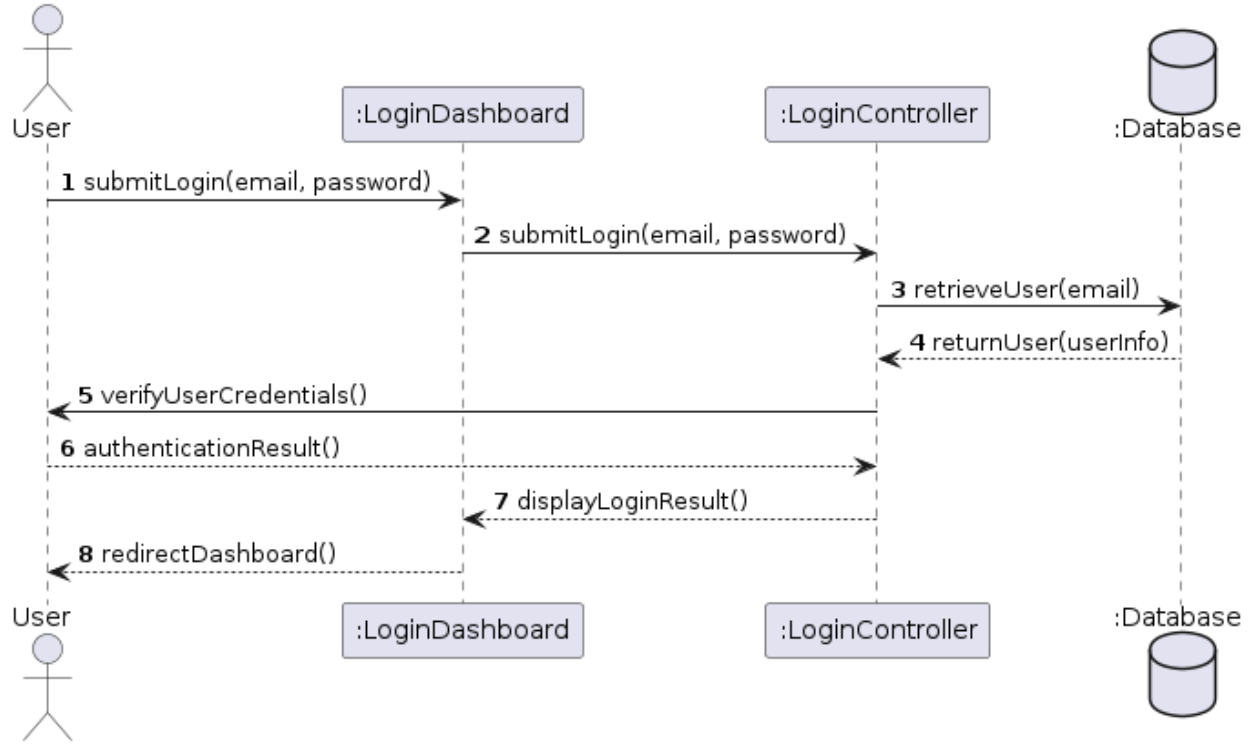

Figure 8: User Login Sequence Diagram

The login process begins when a user enters authentication information through the Login Dashboard. The Login Controller receives the request and retrieves user information from the database. After validating the credentials, the system grants access and redirects the user to the appropriate dashboard according to the user role.

Step	Mitigation
1 [V3]	Authenticate user before accepting login requests.
2 [V3]	Validate login input and prevent unauthorized login attempts.
3[V2]	Encrypt user credentials and protect database communication.
4 [V2]	Protect user information returned from the database.
5 [V3]	Securely verify user credentials and apply password protection.

6 [V2]	Validate authentication results and prevent unauthorized access.
7 [V2]	Secure communication between Login Controller and Dashboard.
8 [V3]	Apply role-based access control before redirecting users.

3.3.2 Create Meeting

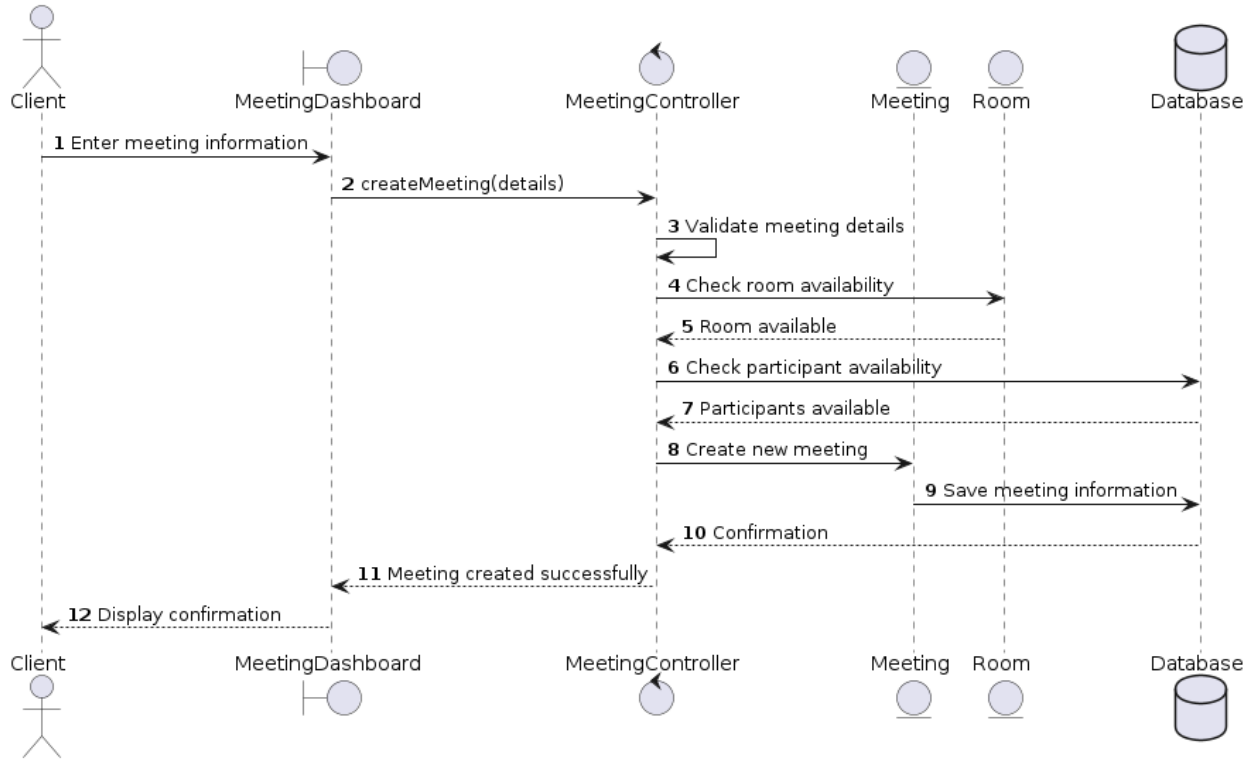

Figure 9: Meeting Creation Sequence Diagram

When a client creates a meeting, the Meeting Dashboard collects meeting information and sends the request to the Meeting Controller. The controller validates the meeting information, checks room availability, verifies participant availability, creates the meeting object, and stores the information in the database. After successful creation, the system returns a confirmation message to the client.

Step	Mitigation
1 [V3]	Authenticate client before allowing meeting creation requests.
2 [V3]	Validate meeting input information and prevent unauthorized requests.
3[V3]	Validate meeting details, including time range, required fields, and business rules.
4 [V2]	Verify room availability and prevent conflicting room reservations.
5 [V2]	Validate room availability response from the Room Module.

6 [V2]	Protect participant information and verify participant availability securely.
7 [V2]	Validate participant availability results returned from the database.
8 [V3]	Verify creator authorization before creating a new meeting.
9 [V2]	Encrypt and securely store meeting information in the database.
10 [V2]	Protect database responses and prevent unauthorized data access.
11[V2]	Secure communication between Meeting Controller and Meeting Dashboard.
12 [V3]	Ensure only authorized users can view meeting confirmation information.

3.3.3 Reserve A Special Room

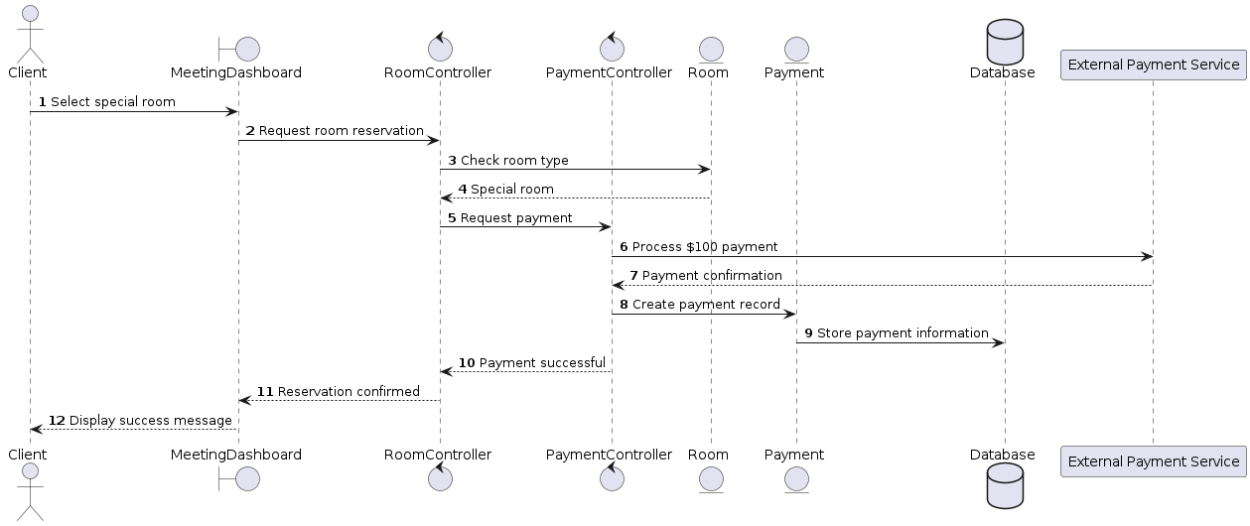

Figure 10: Special Room Reservation Sequence Diagram

When a client reserves a special room, the system verifies the room type and requires payment processing before completing the reservation. The Payment Controller communicates with an external payment service and stores payment records after successful payment verification.

Step	Mitigation
1 [V3]	Authenticate client before allowing special room reservation requests.
2 [V3]	Validate reservation information and verify user authorization.
3[V2]	Verify room information and prevent unauthorized room access.
4 [V2]	Validate room type information returned from the Room Module.

5 [V3]	Authorize payment requests before processing transactions.
6 [V3]	Encrypt payment information during transmission to the external payment service.
7 [V2]	Validate payment confirmation returned from the external payment service.
8 [V2]	Verify payment information before creating payment records.
9 [V3]	Encrypt and securely store payment information in the database.
10 [V2]	Secure communication between Payment Controller and Room Controller.
11[V2]	Protect reservation confirmation data during transmission.
12 [V3]	Ensure only authorized users can access reservation confirmation information.

3.3.4 File Complaint

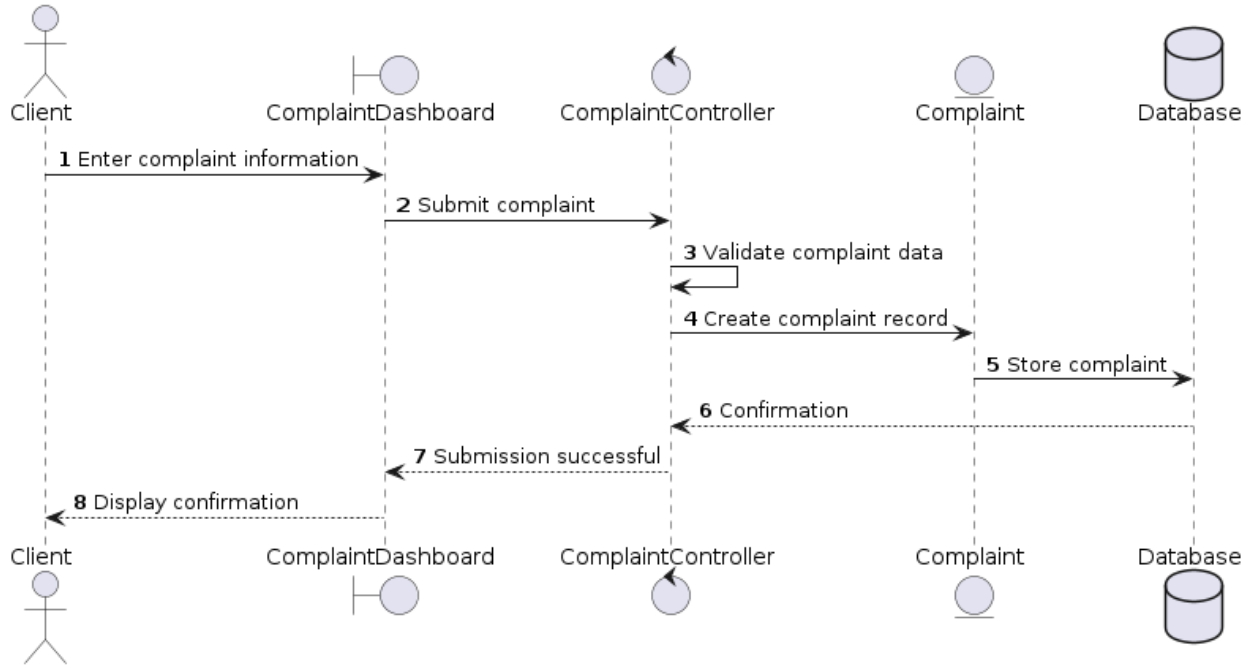

Figure 11: File Complaint Sequence Diagram

The complaint process allows clients to submit issues through the Complaint Dashboard. The Complaint Controller validates the submitted information and creates a complaint record. The complaint information is stored in the database and becomes available for administrators to review and respond.

Step	Mitigation
1 [V3]	Authenticate client before allowing complaint submission.

2 [V3]	Validate complaint input and prevent unauthorized requests.
3[V2]	Perform input validation to prevent malicious or invalid complaint data.
4 [V3]	Verify user authorization before creating complaint records.
5 [V2]	Encrypt and securely store complaint information in the database.
6 [V2]	Protect database responses and prevent unauthorized data disclosure.
7 [V2]	Secure communication between Complaint Controller and Complaint Dashboard.
8 [V3]	Ensure only authorized users can access complaint submission results.

4. Internal Module Design

4.1 Module User and Profile

The User / Profile subsystem class diagram models the MVC structural architecture used to allow Clients to view and update their profile information. The ProfileDashboard serves as the view by displaying the Client's profile information and providing access to profile editing. It communicates with the ProfileController, which retrieves the Client's profile, validates any modifications, and updates the stored information when changes are submitted. Selecting the edit option opens the ProfileEditDashboard, which allows Clients to modify their profile information, submit or cancel changes, display validation errors, and confirm successful updates. The abstract User class defines the common attributes shared by all users, while the Client and Administrator classes inherit these attributes and implement functionality specific to their respective roles. Although both classes inherit from User, only the Client interacts with the Profile subsystem to view and modify profile information.

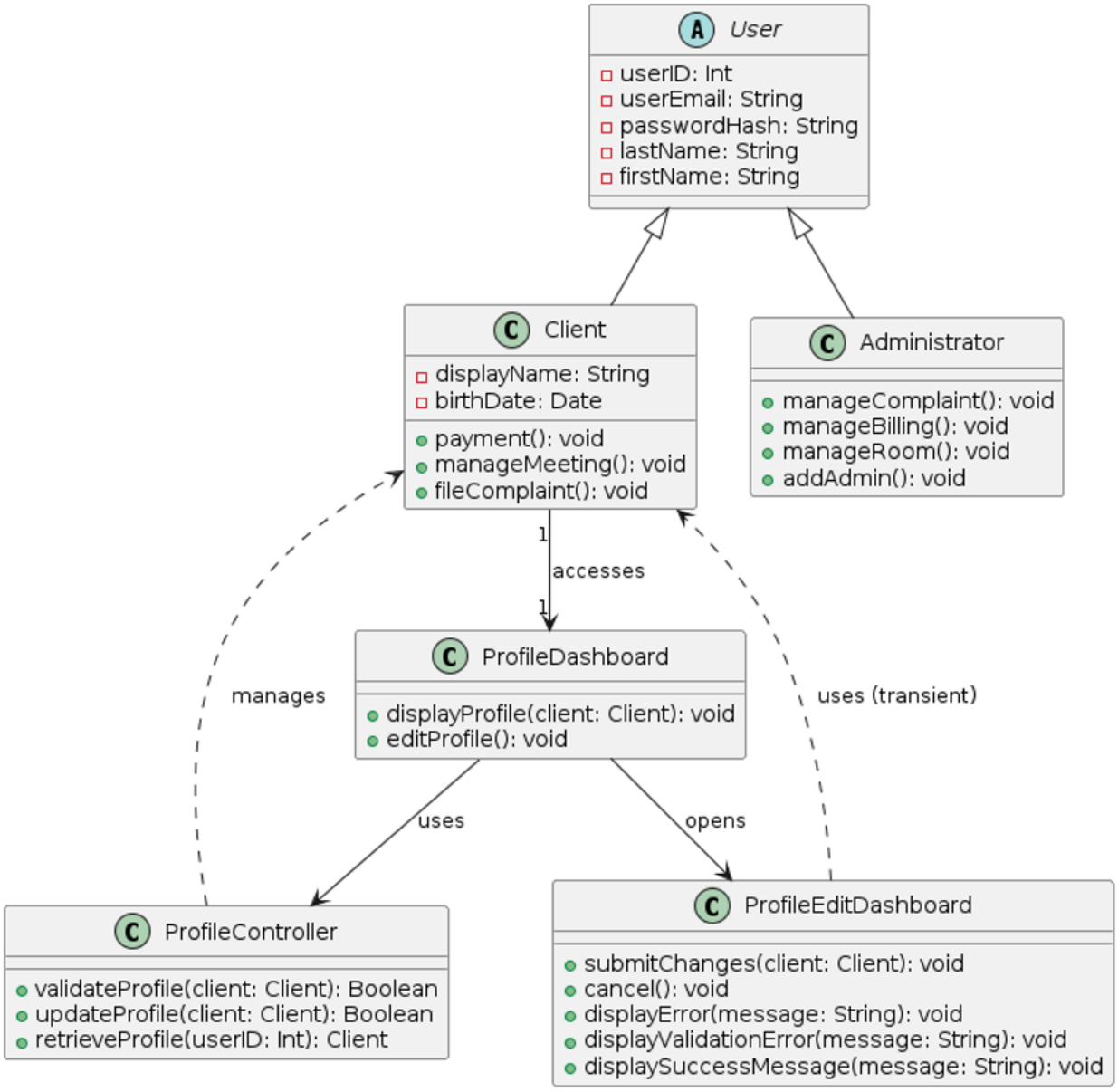

Figure 12: Module User and Profile Class Diagram

4.1.1 Class Administrator

The Administrator class represents the model entity representing the Administrator user. It shows the main responsibilities of the class including managing complaints, managing billing, managing rooms, and creating new administrator accounts.

Class Name	Administrator		
Attribute	**Type**	**Access**	**Description**
N/A	N/A	N/A	N/A
	Visibility	**Return Type**	**Description**

manageComplaint()	Public	void	Retrieves and reviews complaint details, updates the complaint status, and saves the changes to the database.
manageBilling()	Public	void	Retrieves and reviews billing records, updates billing information, and saves the changes to the database.
manageRoom()	Public	void	Selects a room to manage, validates the requested changes, adds or updates room information, and saves the changes.
addAdmin()	Public	void	Enters and validates new administrator information, then creates and saves the administrator account.

Figure 13: Class Administrator Table

4.1.2 Class Client

The Client class represents the model entity representing the Client user. It shows the main responsibilities and attributes within the Client class. This includes the display name and date of birth of the client, as well as making payments, managing meetings, and filing a complaint.

Class Name	Client		
Attribute	**Type**	**Access**	**Description**
displayName	String	Private	Preferred name displayed
birthDate	Date	Private	Client's date of birth
	Visibility	**Return Type**	**Description**
payment()	Public	void	Retrieves and validates payment data, processes the transaction, stores the record, and displayes the payment status.
manageMeeting()	Public	void	Selects and validates meeting information, then updates and saves it to the database.
fileComplaint()	Public	void	Retrieves and validates complaint information, stores the complaint, and notifies the client of its status.

Figure 14: Class Client Table

4.1.3 Class User

The User class is an abstract class that represents the interface for all users. It shows the shared attributes of the abstract class including the user's ID, the user's email, password, last name, and first name that is shared with both the Client class and the Administrator class.

Class Name	*User {Abstract}*		
Attribute	**Type**	**Access**	**Description**
userID	Int	Private	Identifier unique to specific employee
userEmail	String	Private	User’s company email
passwordHash	String	Private	User’s personal password
lastName	String	Private	User’s last name
firstName	String	Private	User’s first name

	Visibility	Return Type	Description
N/A	N/A	N/A	N/A

Figure 15: Class User Table

4.1.4 Class ProfileController

The ProfileController class is the primary controller for the client's profile. It shows the main responsibilities of the ProfileController cless including validating the profile, updating the profile, and retrieving the profile of the client.

Class Name	ProfileController			
Attribute	**Type**		**Access**	**Description**
N/A	N/A		N/A	N/A
	Visibility	**Return Type**	**Description**	
validateProfile(client: Client)	Public	Boolean	Confirms that the client exists in the database and is currently logged in.	
updateProfile(client: Client)	Public	Boolean	Fetches the client's profile applies the new information and saves the changes.	
retrieveProfile(userID: int)	Public	Client	Verifies the user is logged in and returns the profile for the given user ID.	

Figure 16: Class ProfileController Table

4.1.5 Class ProfileDashboard

The ProfileDashboard class is the main dashboard for the client's profile. It shows the main responsibilities of the ProfileDashboard class including displaying the client's profile information and opening the edit profile form.

Class Name	ProfileDashboard		
Attribute	**Type**	**Access**	**Description**
N/A	N/A	N/A	N/A
	Visibility	**Return Type**	**Description**
displayProfile(client: Client)	Public	void	Retrieves the client's ID and profile, then displays the user ID, name, display name, and date of birth.
editProfile()	Public	void	Retrieves and displays the profile edit form.

Figure 17: Class ProfileDashboard Table

4.1.6 Class ProfileEditDashboard

The ProfileEditDashboard is the main dashboard for editing the client's profile. It shows the main responsibilities of the ProfileEEditDashboard class including submitting changes to the profile, cancelling edits, displaying error messages, validation messages, and success messages.

Class Name	ProfileEditDashboard		
Attribute	**Type**	**Access**	**Description**
N/A	N/A	N/A	N/A
	Visibility	**Return**	**Description**

		Type	
submitChanges(client:Client)	Public	void	Submits the updated information to the controller and displays the success or error message based on the result.
cancel()	Public	void	Discards unsaved changes, closes the edit form, and returns to the profile.
displayError(message: String)	Public	void	Displays the given error message.
displayValidationError(message: String)	Public	void	Displays the given validation error message.
displaySuccessMessage(message: String)	Public	void	Displays a success message and returns to the profile.

Figure 18: Class ProfileEditDashboard Table

4.2 Module Complaint Class Diagram

The Complaint subsystem class diagram models the MVC structural architecture used to collect, route, and resolve client complaints within the system. A Client uses fileComplaint() to submit a complaint, which stores details pertaining to the complaint. The ComplaintController validates submissions, manages data flow, and updates the ComplaintViewDashboard and ComplaintEditDashboard interfaces. An Administrator uses manageComplaint() to review submissions, update the status, and provide responses.

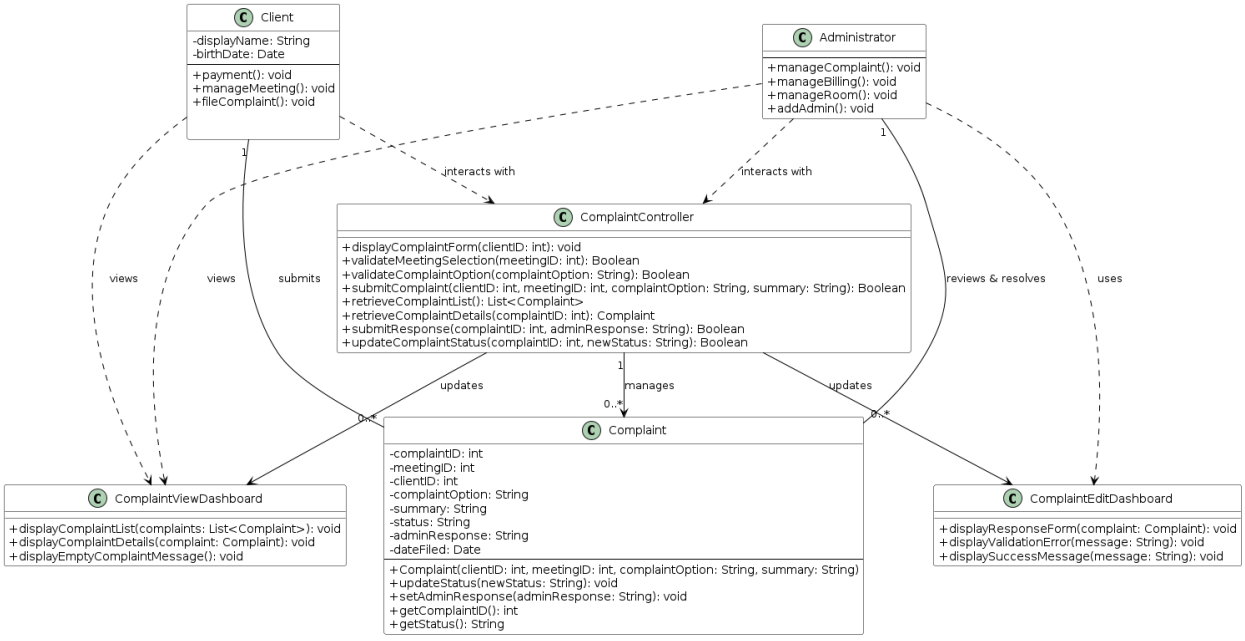

Figure 19: Module Complaint Class Diagram

4.2.1 Class Complaint

The Complaint class serves as the core model entity representing a complaint filed by a client. It encapsulates private complaint details while providing public methods to modify status states and set administrator response.

Class Name	Complaint		
Attribute	**Type**	**Access**	**Description**
complaintID	Int	Private	Identifier unique to the complaint

meetingID	Int	Private	Identifier unique to the meeting
userID	Int	Private	Identifier unique to the user
complaintOption	String	Private	Complaint category selected by the client
summary	String	Private	Text description of the complaint entered by the client
status	String	Private	Current status of the complaint
adminResponse	String	Private	Administrator response to the complaint
dateFiled	Date	Private	Date the complaint was submitted
	Visibility	**Return Type**	**Description**
complaint(userID: int, complaintOption: String, summary: String)	Public	N/A	Initializes a new complaint with the given user, option, and summary, setting its status to Pending and recording the filing date.
updateStatus(newStatus: String)	Public	void	Validates and updates the complaint's status.
setAdminResponse(adminResponse: String)	Public	void	Validates and saves the administrator's response, then marks the complaint as Resolved.
getComplaintID()	Public	int	Returns the complaint's ID.
getStatus()	Public	String	Returns the complaint's status.

Figure 20: Class Complaint Table

4.2.2 Class ComplaintController

The ComplaintController class acts as the central controller mediator within the MVC pattern, coordinating logic and request processing between the view user interfaces and complaint model. It contains public methods to validate forms, process new client submissions, retrieve complaint details, and route administrative updates to storage.

Class Name	ComplaintController		
Attribute	**Type**	**Access**	**Description**
N/A	N/A	N/A	N/A
	Visibility	**Return Type**	**Description**
displayComplaintForm(userID: int)	Public	void	Fetches the client's past meetings and valid complaint categories, then sends them to the form for display.
validateMeetingSelection(meetingID: int)	Public	Boolean	Verifies the meeting exists and was attended by the client.
validateComplaintOption(complaintOption: String)	Public	Boolean	Verifies that a valid complaint category was selected.

submitComplaint(userID: int, meetingID: int, complaintOption: String, summary: String)	Public	Boolean	Validates the meeting and category, then creates and saves a new complaint if valid.
retrieveComplaintList()	Public	List<Complaint>	Verifies administrator privileges and returns the full list of complaints
retrieveComplaintDetails(complaintID: int)	Public	Complaint	Verifies administrator privileges and returns the complaint matching the given ID.
submitResponse(complaintID: int, adminResponse: String)	Public	Boolean	Sets and saves the administrator's response for the given complaint.
updateComplaintStatus(complaintID: int, newStatus: String)	Public	Boolean	Updates and saves the status of the given complaint.

Figure 21: Class ComplaintController Table

4.2.3 Class ComplaintEditDashboard

The ComplaintEditDashboard class represents the primary interface for displaying and performing actions on complaint records by administrators. It provides interface methods to render administrator response forms, display input validation errors, and confirm successful status updates following administrator response submissions.

Class Name	ComplaintEditDashboard		
Attribute	**Type**	**Access**	**Description**
N/A	N/A	N/A	N/A
	Visibility	**Return Type**	**Description**
displayResponseForm(complaint: Complaint)	Public	void	Displays the selected complaint's details and a field for entering the administrator's response.
displayValidationError(message: String)	Public	void	Displays the given error message and stops form submission.
displaySuccessMessage()	Public	void	Displays a confirmation message and refreshes to the complaint list.

Figure 22: Class ComplaintEditDashboard Table

4.2.4 Class ComplaintViewDashboard

The ComplaintViewDashboard class represents the primary interface for displaying complaint records to users. It contains rendering methods designed to format and display the complaint list and selected individual complaints.

Class Name	ComplaintViewDashboard		
Attribute	**Type**	**Access**	**Description**
N/A	N/A	N/A	N/A

displayComplaintList(complaints: List<Complaint>)	Public	void	Displays the complaint ID, meeting ID, option, and status for each complaint in the list.
displayComplaintDetails(complaints: Complaint)	Public	void	Displays the summary, date filed, and available actions for the selected complaint.
displayEmptyComplaintMessage()	Public	void	Displays a notification when no complaints are found.

Figure 23: Class ComplaintViewDashboard

4.3 Module LogIn Class Diagram

The LogIn subsystem class diagram models the MVC structural architecture used to authenticate users before granting access to the PMSS. The LogInDashboard serves as the view by displaying the login form, collecting the user's credentials, and displaying either an error message or a successful login. It communicates with the LogInController, which validates the user's credentials and returns the corresponding User object if authentication is successful. The abstract User class defines the common attributes shared by both Client and Administrator accounts, allowing either user type to authenticate through the same subsystem.

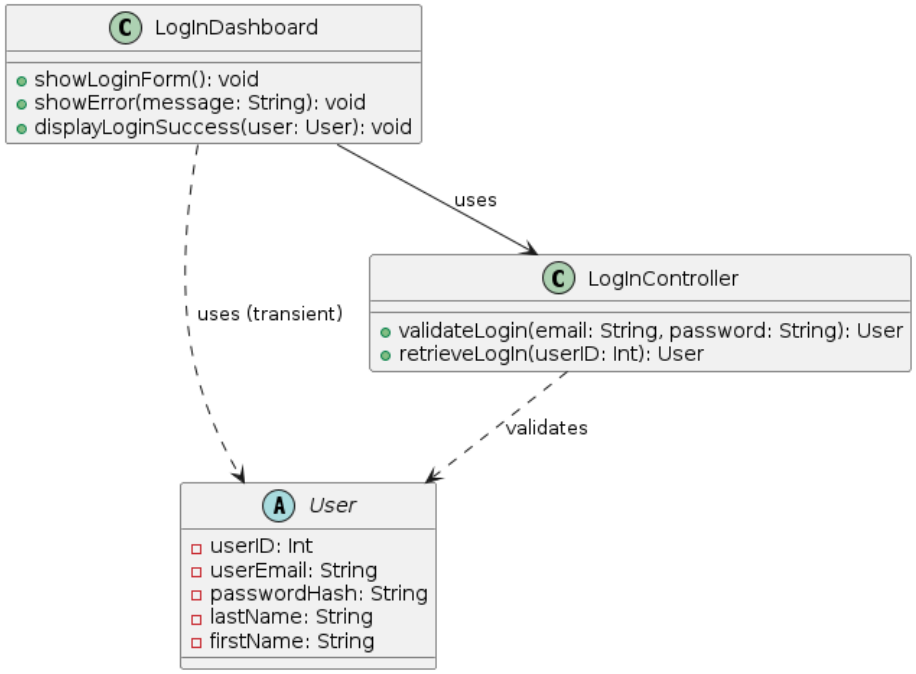

Figure 24: Module LogIn Class Diagram

4.3.1 Class LogInController

The LogInController class is the main controller for the user's login process. It shows the main responsibiilities of the LoogInController class including validating the login credentials and retrieving the login information from the database.

Class Name	LogInController		
Attribute	**Type**	**Access**	**Description**
N/A	N/A	N/A	N/A
	Visibility	**Return Type**	**Description**

validateLogin(email: String, password: String)	Public	User	Validates the given credentials and returns the authenticated user, or null if invalid.
retrieveLogIn(userID: Int)	Public	User	Returns the user matching the given ID, or null if the user does not exist.

Figure 25: Class LoginController Table

4.3.2 Class LogInDashboard

The LogInDashboard class is the main dashboard for the user login process. It shows the main responsibilities of the LogInnDashboard class including displaying the login form, showing error messages, and displaying success messages before directing to the main dashboard.

Class Name	LogInDashboard		
Attribute	**Type**	**Access**	**Description**
N/A	N/A	N/A	N/A
	Visibility	**Return Type**	**Description**
showLoginForm()	Public	void	Displays the login form and waits for submission.
showError(message: String)	Public	void	Displays the given error message.
displayLoginSuccess(user: User)	Public	void	Displays a success message and returns to the profile.

Figure 26: Class LogInDashboard Table

4.4 Module Meeting Class Diagram

The Meeting subsystem class diagram models the MVC structural architecture used to create, schedule, and manage meetings within the system. The MeetingDashboard and MeetingEditDashboard serve as the view, allowing Clients to create, view and edit their meetings through their interface. Both dashboards communicate with the MeetingController, which validates meeting details, checks room availability, verifies participant availability, and coordinates with the Meeting and Room classes to schedule the meeting.

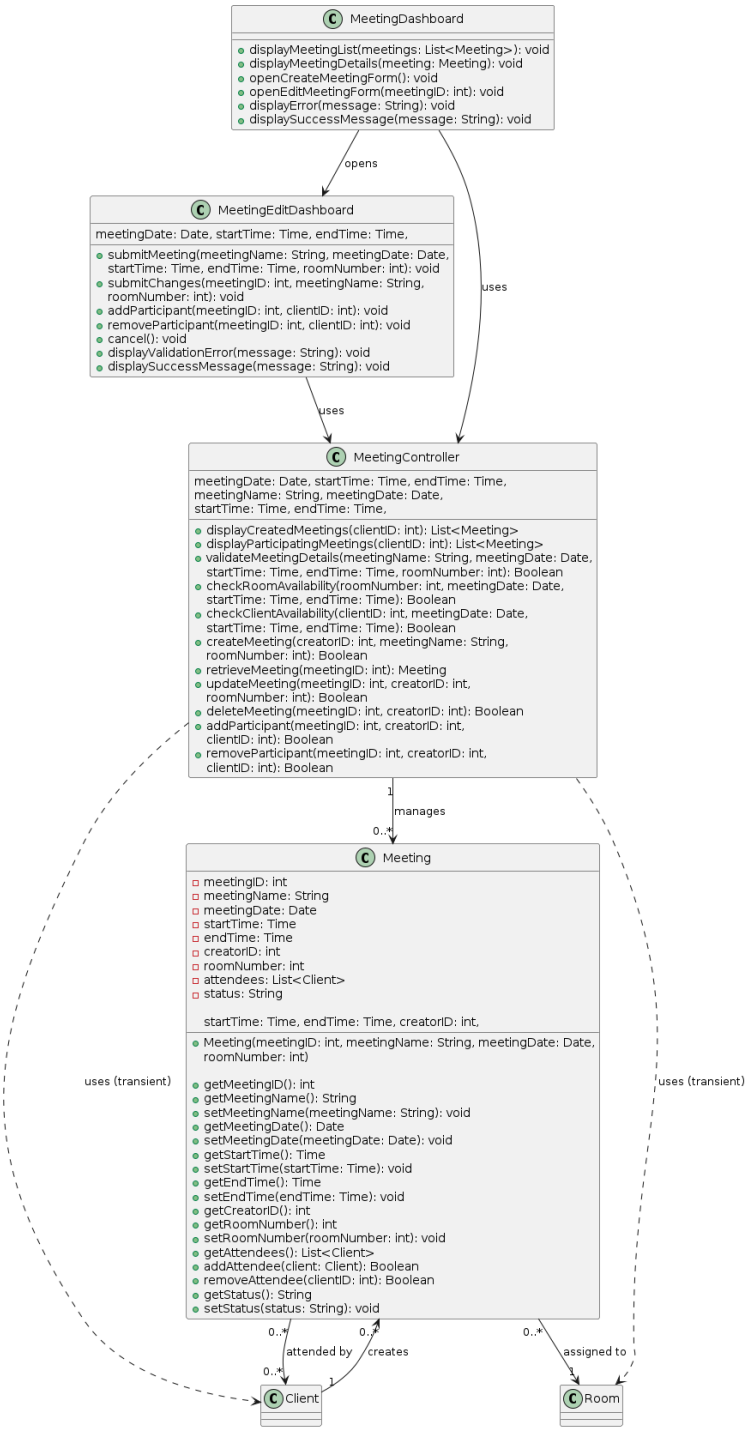

Figure 27: Module Meeting Class Diagram

4.4.1 Class Meeting

The class Meeting represents a meeting created by a client and scheduled in a meeting room.

Class Name	Meeting		
Attribute	**Type**	**Access**	**Description**

meetingID	int	Private	Unique identifier assigned to the meeting
meetingName	String	Private	Name or title of the meeting
meetingDate	Date	Private	Date on which the meeting is scheduled
startTime	Time	Private	Starting time of the meeting
endTime	Time	Private	Ending time of the meeting
creatorID	String	Private	Identifier of the client who created the meeting
roomNumber	String	Private	Number of the room reserved for the meeting
attendees	List<Client>	Private	List of clients attending the meeting
status	String	Private	Meeting status
	Visibility	**Return Type**	**Description**
Meeting(...)	Public	N/A	Initializes a new meeting with the given information, setting up the attendee list and the initial status of Scheduled.
getMeetingID()	Public	int	Returns the meeting's unique identifier.
getMeetingName()	Public	String	Returns the meeting's name.
SetMeetingName (meetingName: String)	Public	void	Sets the meeting's name.
getMeetingDate()	Public	Date	Returns the meeting's scheduled date.
SetMeetingDate (meetingDate: Date)	Public	void	Sets the meeting's scheduled date.
getStartTime()	Public	Time	Returns the meeting's starting time.
SetStartTime (startTime: Time)	Public	void	Sets the meeting's start time.
getEndTime()	Public	Time	Returns the meeting's ending time.
setEndTime(endTime: Time)	Public	void	Sets the meetings ending time.
getCreatorID()	Public	String	Returns the identifier of the meeting's creator.
getRoomNumber()	Public	String	Returns the number of the room assigned to the meeting.
SetRoomNumber (roomNumber: int)	Public	void	Assigns the given room number to the meeting.
getAttendees()	Public	List<Client>	Returns the list of clients attending the meeting.
AddAttendee (client: Client)	Public	Boolean	Adds the given client to the attendee list if not already present.
RemoveAttendee (clientID: int)	Public	Boolean	Removes the given client from the attendee list.
getStatus()	Public	String	Returns the meeting's current status.
SetStatus (status: String)	Public	void	Sets the meeting status.

Figure 28: Class Meeting Table

4.4.2 Class MeetingController

The class MeetingController controls meeting operations, validates scheduling rules, enforces creator permissions, and coordinates the Meeting, Client, and Room classes.

Class Name	MeetingController		
Attribute	**Type**	**Access**	**Description**
N/A	N/A	N/A	N/A
	Visibility	**Return Type**	**Description**
DisplayCreatedMeetings (clientID: int)	Public	List<Meeting>	Returns the list of meetings created by the given client.
DisplayParticipatingMeeting(clientID: int)	Public	List<Meeting>	Returns the list of meetings the given client is attending.
ValidateMeetingDetails(...)	Public	Boolean	Verifies required fields, the one-week scheduling window, buisness hours, and that the end time follows the start time.
CheckRoomAvailability(...)	Public	Boolean	Compares the requested meeting against existing meetings in the selected room to check for conflicts.
CheckClientAvailability(...)	Public	Boolean	Compares the requested meeting against the client's existing meetings to check for scheduling conflicts.
CreateMeeting(...)	Public	Boolean	Validates the meeting details and room availability, then creates and saves the meeting.
RetrieveMeeting(meetingID: String)	Public	Meeting	Returns the meeting matching the given identifier.
UpdateMeeting(...)	Public	Boolean	Validates the requester and updates details, checks for conflicts, and saves the changes.
DeleteMeeting (meetingID: int, creatorID: int)	Public	Boolean	Verifies the requester is the meeting's creator, then cancels or removes the meeting and releases its room.
AddParticipant(...)	Public	Boolean	Verifies the requester and client, checks availability, and adds the client to the meeting.
RemoveParticipant(...)	Public	Boolean	Verifies the requester is the meeting's creator and the client is an attendee, then removes the client.

Figure 29: Class MeetingController Table

4.4.3 Class MeetingDashboard

The class MeetingDashboard displays meeting information and provides navigation to meeting creation and editing interfaces.

Class Name	MeetingDashboard		
Attribute	**Type**	**Access**	**Description**
N/A	N/A	N/A	N/A
	Visibility	**Return Type**	**Description**
DisplayMeetingList	Public	void	Formats and displays the given list of

(meetings: List<Meeting>)			meetings.
DisplayMeetingDetails(meeting: Meeting)	Public	void	Displays the selected meeting's name, date, time, room, creator, attendees, and status.
OpenCreateMeetingForm()	Public	void	Opens the MeetingEditDashboard in create-meeting mode.
OpenEditMeetingForm (meetingID: int)	Public	void	Requests the meeting's information and opens the MeetingEditDashboard with the existing details.
DisplayError(message: String)	Public	void	Displays the given error message.
DisplaySuccessMessage(message: String)	Public	void	Displays a message confirming a successful operation.

Figure 30: Class MeetingDashboard Table

4.4.4 Class MeetingEditDashboard

The class MeetingEditDashboard receives user input for creating and editing meetings and for managing meeting participants.

Class Name	MeetingEditDashboard		
Attribute	**Type**	**Access**	**Description**
N/A	N/A	N/A	N/A
	Visibility	**Return Type**	**Description**
SubmitMeeting(...)	Public	void	Sends the entered meeting information to MeetingController and displays a success or validation error message.
SubmitChanges(...)	Public	void	Sends the modified meeting information to MeetingController and displays the update result.
AddParticipant(meetingID: int, clientID: int)	Public	void	Submits a request to add the given client to the meeting and displays the result.
RemoveParticipant (meetingID: int, clientID: int)	Public	void	Submits a request to remove the given client to the meeting and displays the result
cancel()	Public	void	Discards unsaved information and returns to MeetingDashboard.
DisplayValidationError (message: String)	Public	void	Displays an input or scheduling validation error.
DisplaySuccessMessage (message: String)	Public	void	Displays confirmation that the requested operation was completed.

Figure 31: Class MeetingEditDashboard Table

4.5 Module Register Class Diagram

The Register subsystem class diagram models the MVC structural architecture used to allow users to register for new Client accounts as well as Administrators to create new Administrator accounts. The RegisterDashboard functions as the view, displaying a form with fields for email, password and birthdate. It then calls its RegisterController to validate and create the new user as a Client account. The RegisterAdminDashboard functions the exact same way, instead it is only visible to Administrator accounts, uses a RegisterAdminController, and returns a new Administrator account.

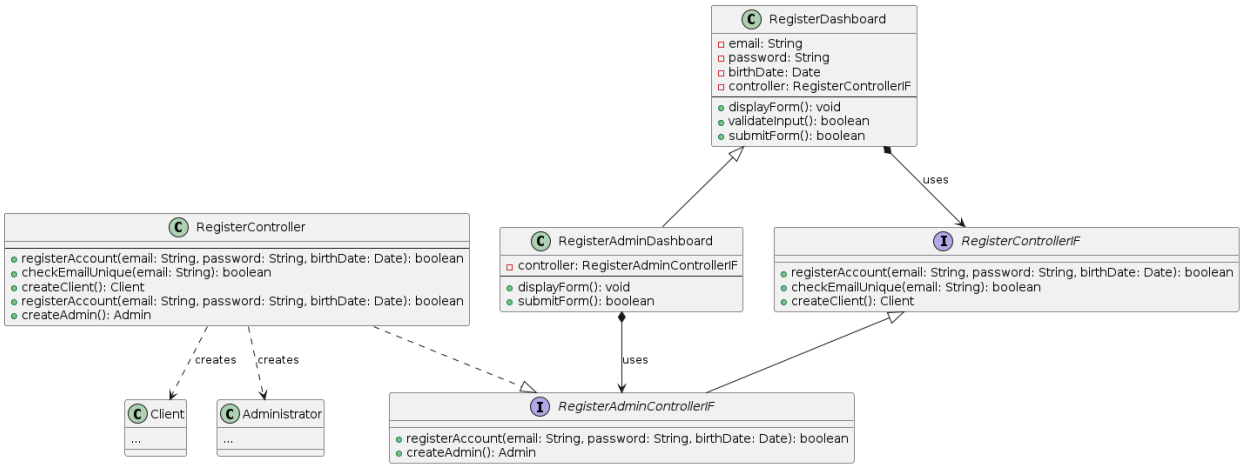

Figure 32: Module Register Class Diagram

4.5.1 Class RegisterDashboard

RegisterDashboard is the view implementation for people to register for new Client accounts. This class is responsible for visually displaying an account registration form, validating the input, and then submitting the form to the RegisterControllerIF object.

Class Name	RegisterDashboard		
Attribute	**Type**	**Access**	**Description**
email	String	Private	Form field for user's email address.
password	String	Private	Form field for user's password.
birthDate	Date	Private	Form field for user's birth date.
controller	RegisterControllerIF	Private	Reference to dashboard's controller object.
Methods	**Visibility**	**Return Type**	**Description**
displayForm()	Public	void	Visually displays form for user.
validateInput()	Public	void	Client-side validation of form fields.
submitForm()	Public	boolean	Submits form fields to controller for account creation.

Figure 33: Class RegisterDashboard Table

4.5.2 Class RegisterAdminDashboard

RegisterDashboard is the view implementation for Administrators to register new Administrator accounts. This class inherits from RegisterDashboard and is responsible for visually displaying an account registration form, validating the input, and then submitting the form to the RegisterAdminControllerIF object.

Class Name	RegisterAdminDashboard		
Attribute	**Type**	**Access**	**Description**
…	…	…	Inherits from RegisterDashboard.
controller	RegisterAdminControllerIF	Public	Overrides reference to RegisterAdminController object.

Methods	Visibility	Return Type	Description
…	…	…	Inherits from RegisterDashboard
displayForm()	Public	void	Overrides form with fields for registering Administrator accounts.
submitForm()	Public	Client	Overrides submit to controller.

Figure 34: Class RegisterAdminDashboard Table

4.5.3 Interface RegisterControllerIF

RegisterControllerIF is the interface subclassed by RegisterAdminControllerIF and referenced by RegisterDashboard. It creates prototypes for the registerAccount, checkEmailUnique, and createClient methods implemented by its concrete subclasses

Class Name	RegisterControllerIF		
Attribute	**Type**	**Access**	**Description**
N/A	N/A	N/A	N/A
Methods	**Visibility**	**Return Type**	**Description**
registerAccount(email: String, password: String, birthDate: Date)	Public	boolean	Driver function for creating a new account, overridden by concrete classes to call either createClient() or createAdmin()
checkEmailUnique(email: String)	Public	boolean	Checks database to see if entered email address has already been claimed or not.
createClient()	Public	Client	Creates and returns new Client account.

Figure 35: Class RegisterControllerIF Table

4.5.4 Interface RegisterAdminControllerIF

RegisterAdminControllerIF is the interface that inherits from RegisterControllerIF and referenced by RegisterAdminDashboard. It overrides the registerAccount method and creates a prototype for the createAdmin method implemented by its concrete subclasses.

Class Name	RegisterAdminControllerIF		
Attribute	**Type**	**Access**	**Description**
N/A	N/A	N/A	N/A
Methods	**Visibility**	**Return Type**	**Description**
…	…	…	…
registerAccount(email: String, password: String,	Public	boolean	Overrides to call createAdmin().

birthDate: Date)			
createAdmin()	Public	Client	Creates and returns new Administrator account.

Figure 36: Class RegisterAdminControllerIF Table

4.5.5 Class RegisterController

RegisterController is the controller implementation for the registration of new accounts. It subclasses RegisterAdminControllerIF (which subclasses RegisterControllerIF) and handles the execution of checking if the email is unique and then creates and returns a Client or Administrator depending on the type of dashboard object is calling it.

Class Name	RegisterController		
Attribute	**Type**	**Access**	**Description**
N/A	N/A	N/A	N/A
Methods	**Visibility**	**Return Type**	**Description**
registerAccount(email: String, password: String, birthDate: Date)	Public	boolean	Evaluates if new account should be a Client or Administrator then calls the relevant function.
checkEmailUnique(email: String)	Public	boolean	Checks if email has been claimed yet or not in database.
createClient()	Public	Client	Creates and returns Client account.
createAdmin()	Public	Administrator	Creates and returns Administrator account.

Figure 37: Class RegisterController Table

4.6 Module Schedule Class Diagram

The Schedule subsystem class diagram models the MVC structural architecture used to display meetings for users. Clients are able to view meetings they are associated with, while Administrators can view all meetings and filter by certain parameters such as timeslot and chosen user. The ScheduleDashboard and AdminScheduleDashboard visually display the list of meetings for Clients and Administrators respectively. Both of these dashboards have references to a ScheduleController object that implements the AdminScheduleControllerIF and ScheduleControllerIF. This SchedulerController will then create a Meeting and add it to its list of tracked meetings. This package also registers the implementation of the Room and SpecialRoom classes for use by Meetings. When a Client reserves a SpecialRoom, the ScheduleController directs the PaymentDashboard to collect payment for the room's fee. The PaymentDashboard displays the payment page, validates the submitted payment information, and displays either a payment confirmation or a payment error before the meeting reservation is finalized.

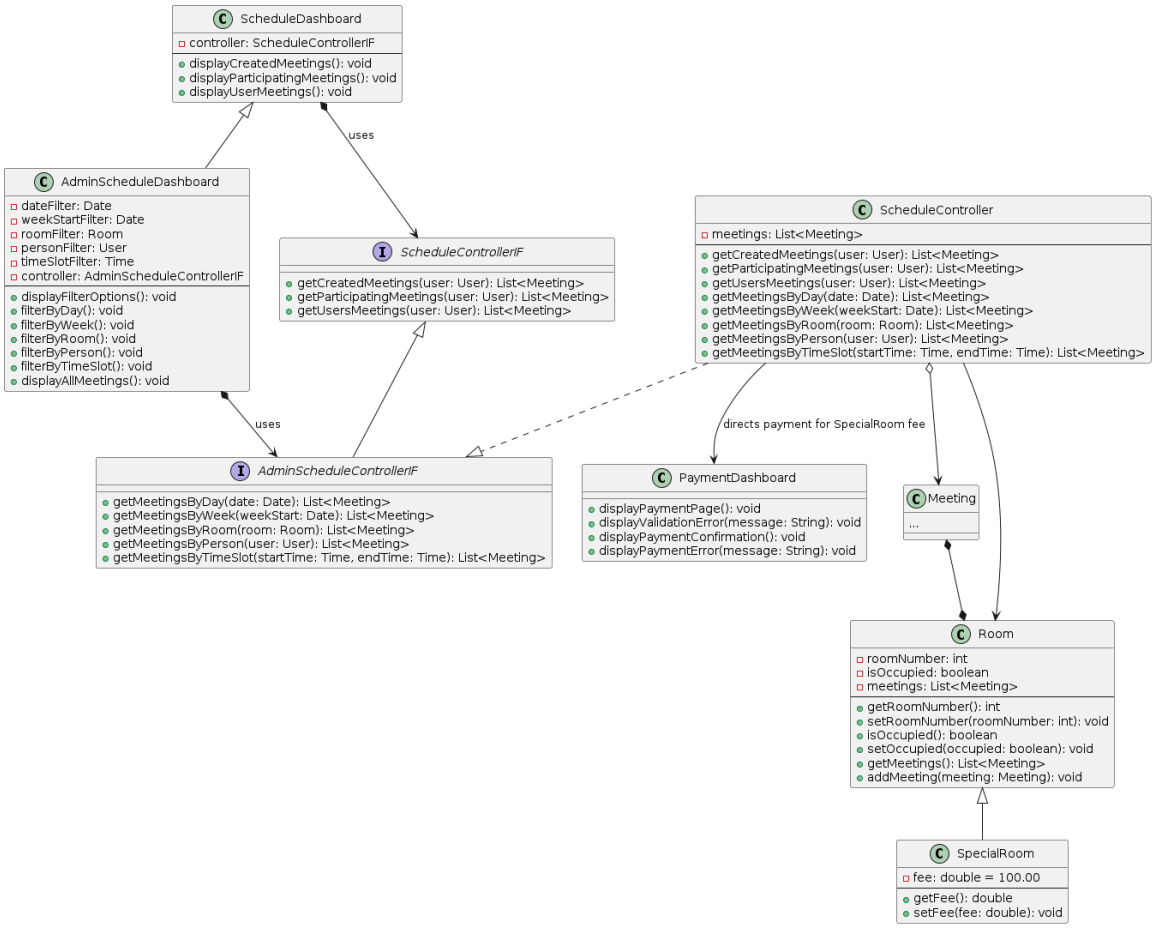

Figure 38: Module Schedule Class Diagram

4.6.1 Class ScheduleDashboard

The ScheduleDashboard class is used for visually displaying the dashboard for a Client. It allows Clients to view the meetings they're associated with, and filter by their created meetings or the meetings they're attending. It has a reference to a ScheduleControllerIF object as its controller.

Class Name	ScheduleDashboard		
Attribute	**Type**	**Access**	**Description**
controller	ScheduleControllerIF	Private	Reference to dashboard's controller.
Methods	**Visibility**	**Return Type**	**Description**
displayCreatedMeetings()	Public	void	Visually displays all of a user's created meetings.
displayParticipatingMeetings()	Public	void	Visually displays all meetings user is participating in that they didn't create.
DisplayUserMeetings()	Public	void	Visually display all meetings a user has created and is participating in.

Figure 39: Class ScheduleDashboard Table

4.6.2 Class AdminScheduleDashboard

The AdminScheduleDashboard class inherits from ScheduleDashboard and is responsible for visually displaying the dashboard for Administrators. In addition to all of the functionality of the ScheduleDashboard, it allows Administrators to view all meetings and filter by date, week, room, user, or timeslot. It overrides its controller reference to be of an AdminScheduleControllerIF object.

Class Name	AdminScheduleDashboard		
Attribute	**Type**	**Access**	**Description**
dateFilter	Date	Private	Filter for a chosen date.
weekStartFilter	Date	Private	Filter for a chosen start of week.
roomFilter	Room	Private	Filter for chosen room.
userFilter	User	Private	Filter for chosen user.
timeSlotFilter	Time	Private	Filter for chosen timeslot
controller	AdminScheduleControllerIF	Private	Override controller to be for Administrator view.
Methods	**Visibility**	**Return Type**	**Description**
…	…	…	Inherited methods from ScheduleDashboard.
displayFilterOptions()	Public	void	Displays filters for dashboard.
filterByDay()	Public	void	Filters dashboard by given date.
filterByWeek()	Public	void	Filters dashboard by given week.
filterByRoom()	Public	void	Filters dashboard by given room.
filterByPerson()	Public	void	Filters dashboard by given user.
filterByTimeSlot()	Public	void	Filters dashboard by given time slot.
displayAllMeetings()	Public	void	Displays all meetings.

Figure 40: Class AdminScheduleDashboard Table

4.6.3 Class PaymentDashboard

The PaymentDashboard class represents the interface for handling special room reservation transactions within the system. It manages the user interface flows for payment collection, providing methods to display billing inputs, render validation errors, and display reservation confirmation details upon successful payment while also managing re-entry attempts and payment errors.

Class Name	PaymentDashboard		
Attribute	**Type**	**Access**	**Description**
N/A	N/A	N/A	N/A
Methods	**Visibility**	**Return Type**	**Description**
displayPaymentPage(): void	Public	void	Displays the payment form with fields for payment method and billing details.

displayValidationError(message: String): void	Public	void	Displays the given validation error and prevents submission until it is corrected.
DisplayPaymentConfirmation: void	Public	void	Displays a success message along with details of the reserved room and scheduled meeting.
DisplayPaymentError(message: String): void	Public	void	Displays the given error and offers options to retry payment, choose another room, or request a refund.

Figure 41: Class PaymentDashboard Table

4.6.4 Interface ScheduleControllerIF

The ScheduleControllerIF interface is referenced by ScheduleDashboard objects to limit the amount of functions they're able to call from the ScheduleController class. This interface is used for Clients and allows them to only perform the functions Clients are given access to.

Interface Name	ScheduleControllerIF		
Attribute	**Type**	**Access**	**Description**
N/A	N/A	N/A	N/A
Methods	**Visibility**	**Return Type**	**Description**
getCreatedMeetings (user: User)	Public	List<Meeting>	Returns list of created meetings for given user.
getParticipatingMeetings(user: User)	Public	List<Meeting>	Returns list of participating meetings for given user.
getUserMeetings(user: User)	Public	List<Meeting>	Returns list of all meetings for a given user.

Figure 42: Interface ScheduleControllerIF Table

4.6.5 Interface AdminScheduleControllerIF

The AdminScheduleControllerIF interface inherits from ScheduleControllerIF and allows Administrators to peform all of the functions a Client can do, as well as ones solely limited to Administrators.

Interface Name	AdminScheduleController		
Attribute	**Type**	**Access**	**Description**
N/A	N/A	N/A	N/A
Methods	**Visibility**	**Return Type**	**Description**
…	…	…	Inherited methods from ScheduleControllerIF.
getMeetingsByDay(date: Date)	Public	List<Meeting>	Returns list of all meetings by given day.

getMeetingsByWeek(weekStart: Date)	Public	List<Meeting>	Returns list of all meetings by given week start.
getMeetingsByRoom(room: Room)	Public	List<Meeting>	Returns list of all meetings by given room.
getMeetingsByPerson(user: User)	Public	List<Meeting>	Returns list of all meetings by given person.
getMeetingsByTimeSlot(startTime: Time, endTime: Time)	Public	List<Meeting>	Returns list of all meetings by given timeslot.

Figure 43: Interface AdminScheduleControllerIF Table

4.6.6 Class ScheduleController

The ScheduleController class is the controller implementation of the Schedule package and is responsible for retrieving meetings given the parameters and sending them to the calling dashboards. It inherits from AdminScheduleControllerIF (which in turn inherits from ScheduleControllerIF) so it implements all of the methods listed out previously. It has a single meetings attribute which is a cached list of the meetings it most recently retrieved, so as to limit unnecessary database calls.

Class Name	ScheduleController		
Attribute	**Type**	**Access**	**Description**
meetings	List<Meeting>	Private	Cached list of displayed meetings.
Methods	**Visibility**	**Return Type**	**Description**
getCreatedMeetings(user: User)	Public	List<Meeting>	Returns list of created meetings for given user.
getParticipatingMeetings(user: User)	Public	List<Meeting>	Returns list of participating meetings for given user.
getUserMeetings(user: User)	Public	List<Meeting>	Returns list of all meetings for a given user.
getMeetingsByDay(date: Date)	Public	List<Meeting>	Returns list of all meetings by given day.
getMeetingsByWeek(weekStart: Date)	Public	List<Meeting>	Returns list of all meetings by given week start.
getMeetingsByRoom(room: Room)	Public	List<Meeting>	Returns list of all meetings by given room.
getMeetingsByPerson(user: User)	Public	List<Meeting>	Returns list of all meetings by given person.
getMeetingsByTimeSlot(startTime:	Public	List<Meeting>	Returns list of all meetings by given timeslot.

Time, endTime: Time)			

Figure 44: Class ScheduleController Table

4.6.7 Class Room

The Room class is the model implementation of the Schedule package and is responsible for storing the data for rooms. These attributes include the room number, whether or not it's currently occupied, and the meetings that reference the room.

Class Name	Room		
Attribute	**Type**	**Access**	**Description**
roomNumber	int	Private	Room's ID.
isOccupied	boolean	Private	True/False value for if room is currently occupied with meeting.
meetings	List<Meeting>	Private	List of meetings that are taking place in this room.
Methods	**Visibility**	**Return Type**	**Description**
getRoomNumber()	Public	int	Evaluates if new account should be a Client or Administrator then calls the relevant function.
setRoomNumber(roomNumber: int)	Public	void	Checks if email has been claimed yet or not in database.
isOccupied()	Public	boolean	Creates and returns Client account.
setOccupied(occupied: boolean)	Public	void	Creates and returns Administrator account.
getMeetings()	Public	List<Meeting>	Getter for list of meetings.
addMeeting(meeting: Meeting)	Public	void	Add new meeting to list of meetings.

Figure 45: Class Room Table

4.6.8 Class SpecialRoom

The SpecialRoom class inherits from the Room class and adds the extra functionality of needing to be paid for. Each SpecialRoom object has a fee associated with it that defaults to $100.00 and the ScheduleController has functionality to differ between Rooms and SpecialRooms, and will force a user to pay in order to reserve a SpecialRoom.

Class Name	SpecialRoom		
Attribute	**Type**	**Access**	**Description**
…	…	…	Inherited attributes from Room.
fee	Double	Private	Fee for reserving a special room, defaults to $100.00
Methods	**Visibilit**	**Return Type**	**Description**

	y		
…	…	…	Inherited methods from Room.
getFee()	Public	boolean	Getter for fee.
setFee(fee: double)	Public	Client	Setter for fee.

Figure 46: Class SpecialRoom Table

5. Team Members Log Sheets

5.1 Victoria Worthington

Date	Task	Duration
07/13/26	Group Meeting	1h 0m
07/16/26	Module Class Diagram – User, Class Employee, Client, Administrator	0h 25m
07/20/26	Module Class Diagram – LogInDashboard, LogInController, Group Meeting	1h 10m
07/22/26	Module Class Diagram – User / Profile (Updated), Login (Updated)	1h 10m
07/23/26	Class Charts - User Class, Login Class, section 2.4, Group Meeting	1h 35m
07/25/26	Section 1.5, 2.4.1, 2.4.2, 2.4.3.1	1h 30m
07/26/26	Section 2.4.3.2 - 2.4.3.8, figure descriptions, document formatting, readjust 3.2 diagram	3h 30m
	Total :	10h 20m

5.2 Samuel Shumake

Date	Task	Duration
07/13/2026	Group Meeting	1h 05m
07/20/2026	Model class diagrams, group meeting	1h 30m
07/22/2026	Register and Schedule class diagrams	1h 00m
07/23/2026	Team Meeting	0h 25m
07/25/2026	Adding Register package diagram and tables, document formatting	2h 30m
07/26/2026	Adding Schedule package diagram and tables, sections 1, 1.1, 2.2, 2.3, and document formatting	3h 30m
	Total :	10h 0m

5.3 Lianjie Sun

Date	Task	Duration
07/13/2026	Group Meeting	1h 0m
07/20/2026	Model Class Diagrams, Group Meeting	1h 30m
07/23/2026	Edit Class Diagrams, Group Meeting	1h 30m

07/26/2026	Section 3	3 h 00m
	Total :	7h 0m

5.4 Tiffany Hart

Date	Task	Duration
07/13/26, 07/23/2026	Group Meeting	1h 25m
07/19/26	Module Class Diagram and Classes: ComplaintController, Complaint, ComplaintEditDashboard, ComplaintViewDashboard, PaymentDashboard	2h 05m
7/24/26	Document formatting, Section 1.2, Edits and captioning for Module Class Diagram and Classes: ComplaintController, Complaint, ComplaintEditDashboard, ComplaintViewDashboard, PaymentDashboard	1h 00m
7/25/26	Section 1.3, 2.1	0h 30m
7/26/26	Document formatting/edits/review	3h 30m
	Total:	8h 30m